Student Name: Mpumelelo Chonco

Student Number: ST10449316

Module: Prog7311 Part 1

Due Date: 27 March 2026

Enterprise Analysis and Architecture (GLMS)

: Global Logistics Management System Report

Introduction

Moving forward begins with leaving behind old tech that does not link well. A stronger web platform must take its place, one ready to grow when needed. In my role leading solutions architecture, I shape how the new Global Logistics Management System comes together. Structure matters here - using tested methods, clear blueprints guide each step. Systems will stretch across regions without breaking, designed that way from day one. Growth pushes change, so the foundation anticipates what comes next.

1) Architectural Framework: TOGAF Strategy

Starting with alignment to company targets, TOGAF stands out as a solid fit for shaping the GLMS. Instead of just organizing ideas as Zachman does, it walks through the steps methodically. Built around **the Architecture** Development Method, it guides design, rollout, oversight, and updates. This approach keeps structure clear across big organizations. Evidence supporting its use comes from The Open Group in 2018.

The TOGAF ADM Phases A to D applied in the GLMS project:

- Starting with Phase A, the first move sets clear boundaries. Instead of scattered spreadsheets, one unified platform takes shape - called the Contract Management Hub along with Service Request Processing. Agreement among top players like the Board, Logistics Managers, and Finance locks in the target: stop broken data flows and avoid rule-breaking slips. Early blueprints sketch out major parts of the system at a broad level. That foundation guides what comes next.

- After finishing Phase A, work shifts toward shaping how operations function day to day. Mapping out actions comes next - especially steps tied to submitting a service request when a live contract exists. Who does what matters here; roles like Logistics Manager, Client, and Finance Officer enter the picture. Each person connects to others in specific ways, forming clear lines of activity. The flow must match real-world procedures companies rely on. One example is updating contract states automatically instead of by hand. Logic built into the system reflects actual working conditions, nothing more, nothing less. Rules guiding tasks get embedded, so actions follow expected patterns without deviation.

- Now comes Phase C - shaping the data and app layouts for the GLMS. It matters a lot. The setup includes key pieces like Contract, Client, ServiceRequest, Invoice. These form the backbone of information flow. Instead of just listing parts, we map how they connect. Structure guides function here. Components fit together based on earlier design choices from Section 3. One-part talks to currency tools outside the system. That link must work smoothly. Logic inside stays clear because patterns set the rules. How things talk, where data lives, which piece does what - it all gets decided now. Decisions made now shape behaviour later. Nothing flashy, just solid groundwork.
- Out here, servers run inside containers using Docker - flexible pieces that keep things light. Clusters distribute incoming web traffic throughout hardware sets, so

performance stays high during peak usage. Modular services operate independently but connect efficiently to solve complex technical problems fast. Built this way, growing later feels natural instead of forced. The setup answers today's needs while making room for what comes next.
Starting with the ADM means every choice - how databases are shaped or how containers go live - ties directly to what the business needs. That connection keeps the system grounded in real tasks it must handle, reducing chances it will miss the mark once running. Josyula noted this link matters most when stakes are high.

2) Design Selection: Gang of Four Patterns

For long-term ease in updates and testing, a few GoF patterns will go into the C prototype. Not just one approach fits every part of the situation shown. Where connections grow tangled, relying only on Singleton falls short. Instead, mixing strategies makes structure clearer. Each choice responds directly to how pieces interact. Simplicity stays possible by avoiding rigid single-method fixes.

1. Strategy Pattern (Behavioural)

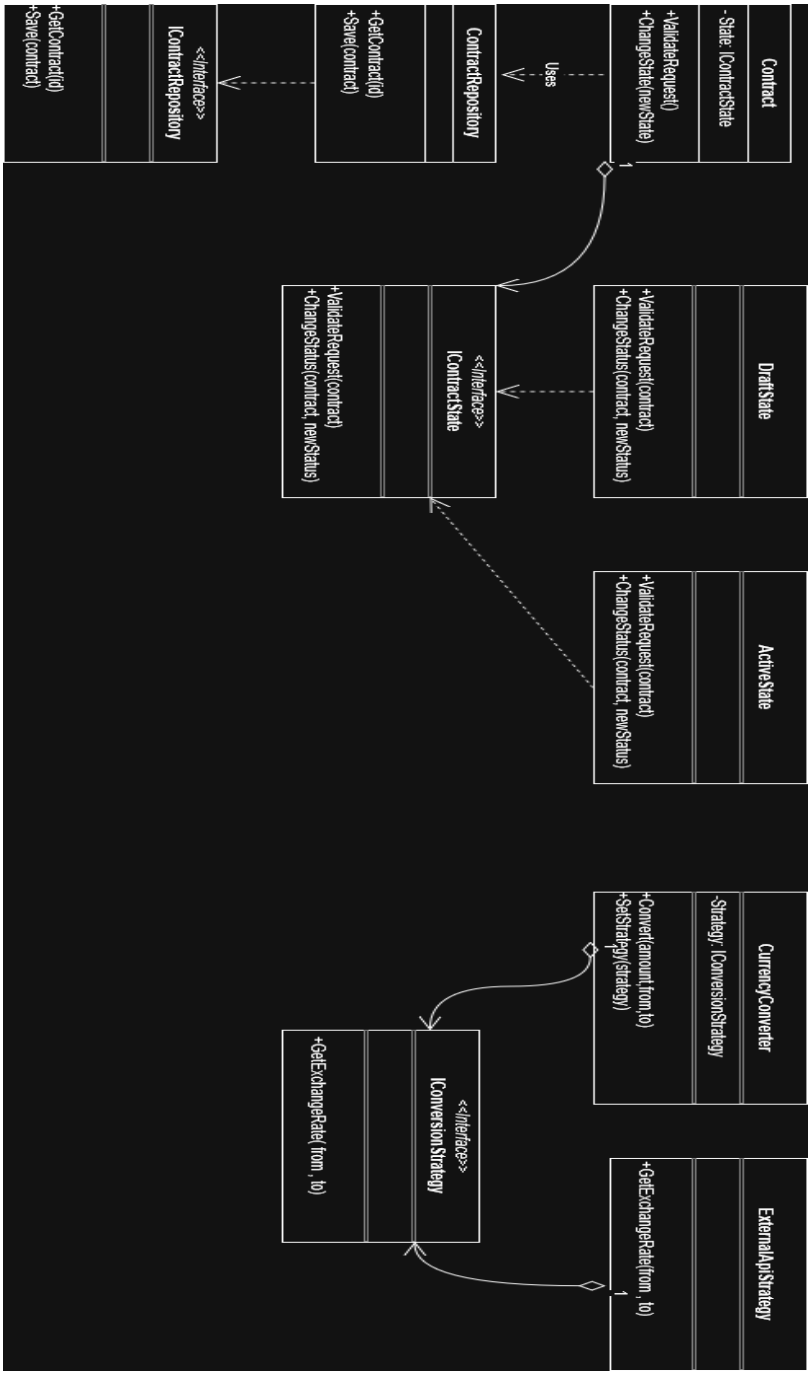
- One way money moves across borders is through instant conversion between currencies. Sometimes the tool that gives those rates might get swapped out later on. A switch could happen if the current service stops being enough. Free options may not last forever. Another system might take its place down the line. What feeds the numbers today might not tomorrow. Updates like these can come quietly. Rate sources tend to shift without warning. Replacement services often step in when limits hit. The flow stays the same even if the source changes.
- Switching how money gets converted happens inside CurrencyConverter by plugging in different tools. Each tool follows the ICurrencyConversionStrategy rulebook but works its own way. Need a live market rate? Plug in ExternalApiStrategy on the fly. Prefer locked numbers? Swap it out for FixedRateStrategy instead. The Invoice part stays untouched no matter which one runs. Swapping pieces never forces changes elsewhere.

2. State Pattern (Behavioural)

- Context: The "Contract Management Hub" requires automated status tracking (Draft, Active, Expired, On Hold).
- Holding a link to an IContractState object, the Contract class puts each condition - such as DraftState or ActiveState - in charge of its own rules. When ValidateServiceRequest() runs, it leans on whatever state is active at that moment. Decisions unfold naturally instead of piling up inside long conditional blocks. Logic stays clean because validation happens only when the contract reaches ActiveState. Rules shift without clutter thanks to delegation. The structure adapts quietly behind the scenes.

3. Repository Pattern (Structural)

- A single storage hub helps meet current needs. It also prepares for what comes next - switching databases later, if necessary. Flexibility builds in quietly. Systems grow without shouting about it. Change hides in plain sight.
- A different way to handle data begins with an IContractRepository setup. Instead of linking directly to a database, the system leans on this middle layer. Swapping storage methods becomes easier because of that separation. Even adding speed improvements, like handling fifty thousand added contracts through temporary memory holds, won't disrupt core operations. Business rules stay untouched when backend details shift.



3) Non-Functional Scalability and Requirements

For the system to handle big business needs, these extra rules beyond basic functions have been laid out.

Availability

Almost every second counts when systems go down - especially since TechMove handles global shipping logistics. Downtime means money slips away, rules get broken. Ninety-nine point nine percent availability? That is non-negotiable outside scheduled updates. Running on clusters built with Docker Swarm or similar tools makes it possible. When one main server stops working, another jumps in right away - no human needed to press buttons. Built-in redundancy ensures operations continue like nothing happened. Failover works quietly behind the scenes, always ready. Reliability isn't added later; it lives at the core of how everything runs.

Interoperability

Expose a RESTful API so outside partners - like customs databases - can connect, while internal finance tools link up too. Data moving back and forth follows standard patterns, mainly JSON, clearly laid out using OpenAPI or Swagger docs. The setup supports two-way communication for financial links, letting invoice and contract statuses be checked by connected platforms. Bidirectional flow means queries go both ways without breaking form.

Handling fifty thousand new contracts

A sudden spike in traffic might follow if another company joins forces with TechMove. To manage heavier demands, changes will roll out across systems slowly. Instead of collapsing under pressure, components can scale outward when needed. When workloads climb, extra resources activate automatically. Growth won't stall because design choices now allow breathing room. Even during peaks, response times stay steady thanks to smarter routing. Capacity expands not all at once but step by step

- More machines join instead of swapping for one big machine. Each handles work without saving data between steps. Traffic spreads using tools like Nginx at the front. Requests move smoothly even when demand jumps after recent deals. Any extra load gets shared naturally among them. Machines come and go without breaking flow. Performance stays steady under heavier use. New capacity fits in quietly behind the scenes.

- Handling fifty thousand new contracts might slow things down right where the data lives. Instead of tying everything directly to one storage setup, using the Repository Pattern pulls that part aside like changing lanes smoothly. Splitting the Contracts table? That becomes possible by spreading it over several databases, guided by something like ClientRegion. One server does not get swamped because work gets handed around instead. Load shifts across machines, keeping reads and writes moving without tripping over themselves.

- When lots of people need contract info fast, hitting the database every time slows things down. So instead, active contracts along with client records get stored temporarily in a shared memory space like Redis. This setup means common requests pull from quick storage rather than waiting on slow queries. As new entries climb toward fifty thousand, repeated checks skip long waits by using saved snapshots. Speed stays high because lookups avoid constant back-and-forth trips to main storage. Fewer delays happen exactly when users expect instant replies.

Conclusion

A fresh look at the setup for the Global Logistics Management System lines up with what the board wanted - smooth connections and room to grow. Moving forward with TOGAF ADM brings order, shifting old routines into a full company-wide platform without chaos. Patterns like State, Strategy, and Repository hold the code together, keeping it clean and strong while making sure rules such as checking live contracts work right and money systems link easily. When demand jumps - a sudden rise to 50,000 contracts - the design handles pressure through splitting loads, spreading data, and smart storage, letting TechMove Logistics stay fast, steady, and always online.

Referencing

Wan, Z., Zhang, Y., Xia, X., Jiang, Y. and Lo, D., 2023, November. Software architecture in practice: Challenges and opportunities. In *Proceedings of the 31st ACM joint European software engineering conference and symposium on the foundations of software engineering* (pp. 1457-1469).
(opened: 23 March 2026)

Qurratuaini, H., 2018, September. Designing enterprise architecture based on TOGAF 9.1 framework. In *IOP Conference Series: Materials Science and Engineering* (Vol. 403, No. 1, p. 012065). IOP Publishing.
(opened : 24 March 2026)

Sadalage, P.J. and Fowler, M., 2013. *NoSQL distilled: a brief guide to the emerging world of polyglot persistence*. Pearson Education.
(opened: 25 March 2026)

Kotusev, S., 2018. TOGAF version 9.2: what's new. British Computer Society (BCS), URL: https://publications.opengroup.org/c182

(opened: 26 March 2026)